

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import LabelEncoder
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.metrics import accuracy_score, classification_report

```

```

1 df = pd.read_csv("train.csv")
2
3 df.head()

```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Status
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```

1 print("Rows and Columns:", df.shape)
2
3 print("\nColumn Names:")
4 print(df.columns)
5
6 print("\nMissing Values:")
7 print(df.isnull().sum())

```

Rows and Columns: (614, 13)

Column Names:
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'],
dtype='object')

Missing Values:

```

Loan_ID      0
Gender       13
Married       3
Dependents   15
Education     0
Self_Employed 32
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount    22
Loan_Amount_Term 14
Credit_History 50
Property_Area  0
Loan_Status   0
dtype: int64

```

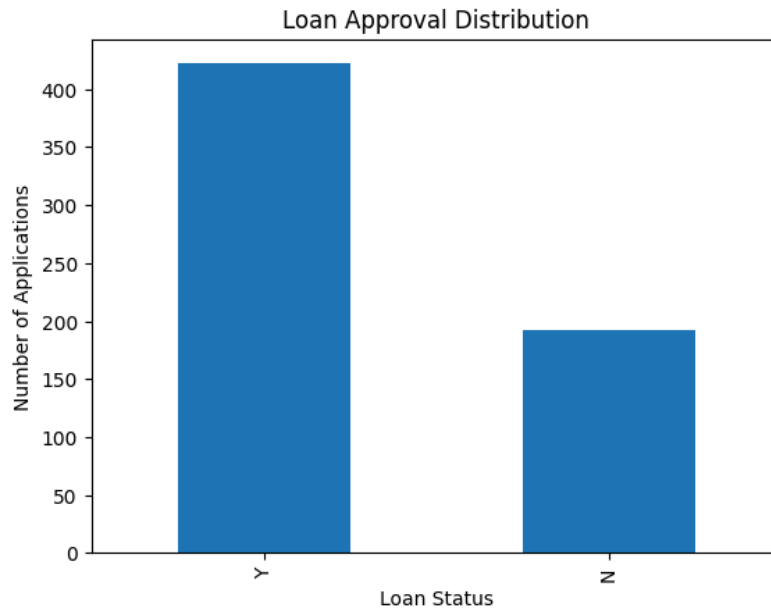
Loan Approval Distribution

```

1 print(df["Loan_Status"].value_counts())
2
3 df["Loan_Status"].value_counts().plot(kind="bar")
4
5 plt.title("Loan Approval Distribution")
6 plt.xlabel("Loan Status")
7 plt.ylabel("Number of Applications")
8 plt.show()

```

```
Loan_Status
Y    422
N    192
Name: count, dtype: int64
```



Handle Missing Values

```
1 df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])
2 df["Married"] = df["Married"].fillna(df["Married"].mode()[0])
3 df["Dependents"] = df["Dependents"].fillna(df["Dependents"].mode()[0])
4 df["Self_Employed"] = df["Self_Employed"].fillna(df["Self_Employed"].mode()[0])
5 df["Credit_History"] = df["Credit_History"].fillna(df["Credit_History"].mode()
  [0])
6
7 df["LoanAmount"] = df["LoanAmount"].fillna(df["LoanAmount"].median())
8 df["Loan_Amount_Term"] = df["Loan_Amount_Term"].fillna(df["Loan_Amount_Term"].
  median())
9
10 print(df.isnull().sum())
```

```
Loan_ID          0
Gender           0
Married          0
Dependents       0
Education        0
Self_Employed    0
ApplicantIncome  0
CoapplicantIncome 0
LoanAmount       0
Loan_Amount_Term 0
Credit_History  0
Property_Area    0
Loan_Status      0
dtype: int64
```

Converting Text to Numbers

```
1 le = LabelEncoder()
2
3 for column in df.columns:
4     if df[column].dtype == "object":
5         df[column] = le.fit_transform(df[column])
6
7 df.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Status
0	0	1	0	0	0	0	5849	0.0	NaN	
1	1	1	1	1	0	0	4583	1508.0	128.0	
2	2	1	1	0	0	1	3000	0.0	66.0	
3	3	1	1	0	1	0	2583	2358.0	120.0	
4	4	1	0	0	0	0	6000	0.0	141.0	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Preparing Features

```
1 X = df.drop(["Loan_ID", "Loan_Status"], axis=1)
2
3 y = df["Loan_Status"]
4
5 print(X.shape)
6 print(y.shape)
```

```
(614, 11)
(614,)
```

splitting data

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     X,
3     y,
4     test_size=0.2,
5     random_state=42
6 )
```

```
1 model = RandomForestClassifier(random_state=42)
2
3 model.fit(X_train, y_train)
4
5 y_pred = model.predict(X_test)
```

```
1 accuracy = accuracy_score(y_test, y_pred)
2
3 print("Model Accuracy:", round(accuracy * 100, 2), "%")
4
5 print("\nClassification Report:")
6 print(classification_report(y_test, y_pred))
```

Model Accuracy: 78.86 %

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.47	0.61	43
1	0.77	0.96	0.86	80
accuracy			0.79	123
macro avg	0.82	0.71	0.73	123
weighted avg	0.80	0.79	0.77	123

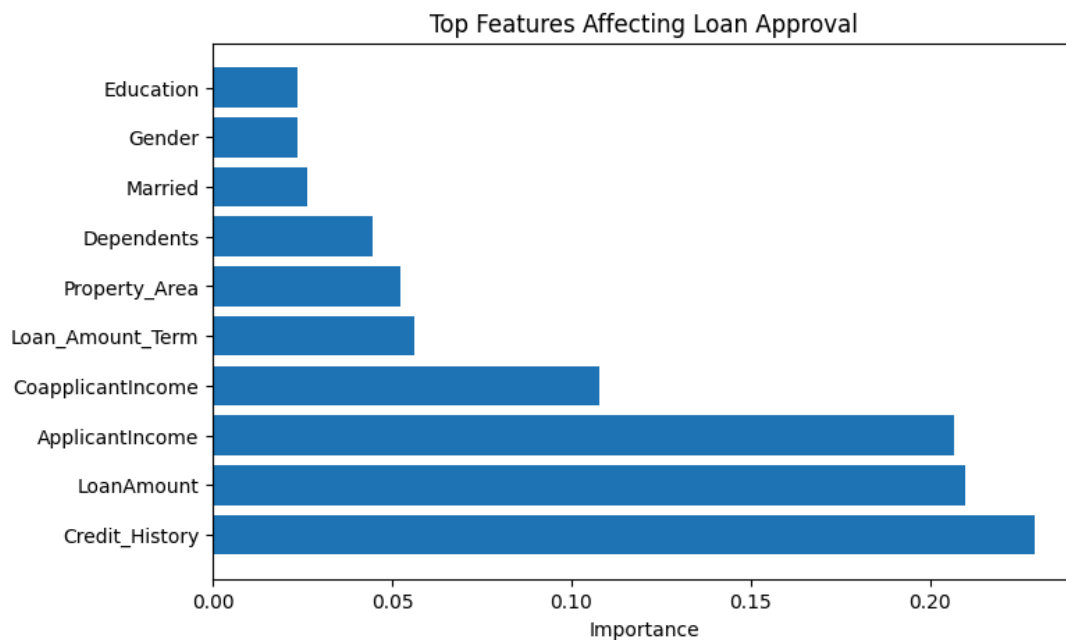
```
1 importance = pd.DataFrame({
2     "Feature": X.columns,
3     "Importance": model.feature_importances_
4 })
5
6 importance = importance.sort_values(
7     by="Importance",
8     ascending=False
9 )
10
11 print(importance.head(10))
```

	Feature	Importance
9	Credit_History	0.229122
7	LoanAmount	0.209715
5	ApplicantIncome	0.206615
6	CoapplicantIncome	0.107859
8	Loan_Amount_Term	0.056034
10	Property_Area	0.052357
2	Dependents	0.044506
1	Married	0.026423
0	Gender	0.023673
3	Education	0.023661

```

1 top10 = importance.head(10)
2
3 plt.figure(figsize=(8,5))
4
5 plt.barh(
6     top10["Feature"],
7     top10["Importance"]
8 )
9
10 plt.title("Top Features Affecting Loan Approval")
11
12 plt.xlabel("Importance")
13
14 plt.show()

```

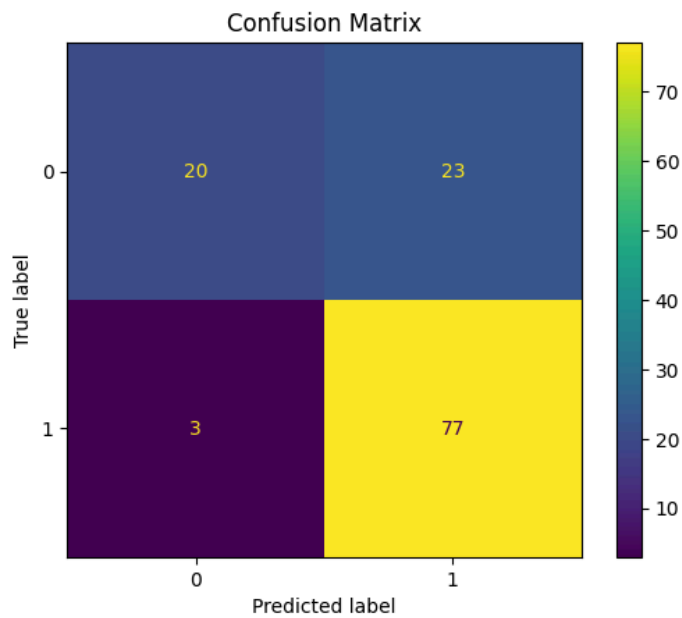


Confusion Matrix

```

1 from sklearn.metrics import ConfusionMatrixDisplay
2
3 ConfusionMatrixDisplay.from_estimator(
4     model,
5     X_test,
6     y_test
7 )
8
9 plt.title("Confusion Matrix")
10
11 plt.show()

```



Loan Approval Rate

```
1 approval_rate = (  
2     df["Loan_Status"].mean()  
3 ) * 100  
4  
5 print(  
6     f"Loan Approval Rate: {approval_rate:.2f}%"  
7 )
```

Loan Approval Rate: 68.73%

Correlation Heatmap

```
1 import seaborn as sns  
2  
3 plt.figure(figsize=(10,8))  
4  
5 sns.heatmap(  
6     df.corr(),  
7     cmap="coolwarm"  
8 )  
9  
10 plt.title("Correlation Heatmap")  
11  
12 plt.show()
```

